

INFORME DE MANTENIMIENTO DE SOFTWARE

Proyecto: Missions CRUD – Sistema de Registro de Misiones Espaciales

Nombre: Maldonado Shirley

Asignatura: Construcción de Software

Facultad: Facultad de Ingeniería de Sistemas – EPN

Tema: Mantenimiento de Software aplicado a un CRUD

Sistema desarrollado: Missions CRUD

Tecnologías: Node.js, Express, SQLite, JavaScript, ESLint, Node:test, OpenAPI y Postman

1. Introducción

El presente informe describe el proceso de mantenimiento aplicado al sistema **Missions CRUD**, una aplicación web orientada al registro y administración de misiones espaciales. El sistema permite crear, consultar, actualizar y eliminar misiones mediante una API REST desarrollada con Node.js y Express, utilizando SQLite como base de datos.

El objetivo principal del mantenimiento fue mejorar la calidad del sistema, corregir incidencias técnicas, adaptar el software a nuevas condiciones de ejecución, agregar funcionalidades de valor y prevenir fallos futuros. Para ello se aplicaron los cuatro tipos fundamentales de mantenimiento de software: correctivo, adaptativo, perfectivo y preventivo.

Además, el CRUD fue integrado con el **EPN Event Manager**, permitiendo registrar eventos de tipo CREATE, UPDATE, DELETE y QUERY cada vez que se realiza una acción en el sistema.

2. Descripción General del Sistema

El sistema **Missions CRUD** permite gestionar información relacionada con misiones espaciales. Cada misión puede contener datos como nombre, agencia espacial, tipo de misión, vehículo, fecha de lanzamiento, estado, tripulación y descripción.

Las operaciones principales son:

Operación	Endpoint	Descripción
Crear	POST /missions	Registra una nueva misión espacial
Consultar	GET /missions	Lista las misiones registradas
Consultar por ID	GET /missions/:id	Obtiene una misión específica
Actualizar	PUT /missions/:id	Modifica los datos de una misión
Eliminar	DELETE /missions/:id	Elimina una misión registrada

El sistema utiliza una base de datos SQLite para conservar la información, lo que permite que las misiones permanezcan guardadas incluso después de cerrar o reiniciar el servidor.

3. Diagnóstico Inicial

Durante la revisión del CRUD se identificaron varias oportunidades de mejora relacionadas con la calidad, seguridad, trazabilidad y mantenibilidad del sistema.

Entre las principales incidencias se encontraron:

Tipo de problema	Incidencia detectada
Trazabilidad	No existía un registro formal de auditoría para las operaciones realizadas
Configuración	Algunos valores estaban definidos directamente en el código
Seguridad	Los endpoints no contaban con protección mediante API-Key
Persistencia	Era necesario asegurar que los datos del CRUD se guarden correctamente en SQLite
Calidad	Faltaban pruebas automatizadas y documentación formal de endpoints
Prevención	Era necesario validar entradas inválidas, scripts maliciosos y posibles inyecciones SQL
Manejo de errores	Se requería un tratamiento centralizado de excepciones

A partir de este diagnóstico se aplicaron los cuatro mantenimientos solicitados.

4. Mantenimiento Correctivo

4.1 Incidencia Técnica Encontrada

El sistema no contaba con un mecanismo adecuado de auditoría ni con un manejo centralizado de errores. Esto dificultaba identificar qué operación se realizó, cuándo ocurrió, desde qué ruta fue ejecutada y si existió algún error durante el proceso.

Sin esta trazabilidad, el diagnóstico de fallos se vuelve más complejo, especialmente cuando el CRUD se integra con otros sistemas como el EPN Event Manager.

4.2 Intervención Realizada

Se implementó un sistema de **logs estructurados de auditoría**, registrando información relevante de cada operación realizada en el sistema.

Los logs incluyen:

- Acción ejecutada: CREATE, UPDATE, DELETE o QUERY.
- Método HTTP.
- Ruta utilizada.
- Dirección IP.
- Fecha y hora en formato ISO 8601.
- Nivel del log: INFO, WARN o ERROR.
- Archivo de auditoría logs/audit.log.

También se agregó un middleware centralizado para manejar errores del servidor y responder de manera controlada ante fallos inesperados.

4.3 Justificación

Este cambio corresponde a **mantenimiento correctivo** porque permite corregir una deficiencia funcional relacionada con la falta de trazabilidad y control de errores. El sistema ahora puede registrar sus operaciones y facilitar la identificación de fallos durante la ejecución.

4.4 Resultado Obtenido

El sistema ahora cuenta con mayor capacidad de diagnóstico. Cada operación importante queda registrada, lo cual permite verificar el comportamiento del CRUD y detectar problemas con mayor facilidad.

5. Mantenimiento Adaptativo

5.1 Incidencia Técnica Encontrada

El sistema necesitaba adaptarse a nuevas condiciones de ejecución y seguridad. Algunos valores como el puerto, la ruta de la base de datos, la ruta de logs y la activación de eventos requerían mayor flexibilidad.

Además, se necesitaba proteger los endpoints mediante una clave de acceso, debido a que el CRUD puede formar parte de una integración con otros sistemas.

5.2 Intervención Realizada

Se implementó configuración externa mediante variables de entorno y un archivo .env.example.

Variables configurables:

Variable	Descripción
PORT	Puerto donde corre el CRUD
API_KEY_HEADER	Nombre de la cabecera usada para autenticación
FIS_EPN_API_KEY	Clave requerida para consumir la API
REQUIRE_API_KEY	Activa o desactiva la validación de API-Key
MISSIONS_DB_PATH	Ruta de la base de datos SQLite

LOG_FILE_PATH	Ruta del archivo de auditoría
SEND_EVENTS	Activa o desactiva el envío de eventos
CORS_ORIGINS	Configuración de orígenes permitidos

También se implementó un middleware requireApiKey, que valida la cabecera X-FIS-EPN-KEY.

Si la clave es válida, la petición continúa. Si no existe o es incorrecta, el sistema responde con error 401 Unauthorized.

5.3 Justificación

Este cambio corresponde a **mantenimiento adaptativo** porque el sistema fue ajustado para funcionar bajo nuevas reglas de entorno, configuración y seguridad. La aplicación ya no depende únicamente de valores escritos directamente en el código, sino que puede adaptarse a distintos escenarios de despliegue.

5.4 Resultado Obtenido

El CRUD ahora es más flexible, configurable y seguro. Puede ejecutarse en diferentes ambientes sin modificar el código fuente y permite proteger sus endpoints mediante API-Key.

6. Mantenimiento Perfectivo

6.1 Incidencia Técnica Encontrada

El sistema necesitaba mejorar su funcionalidad, documentación y facilidad de uso. Aunque el CRUD realizaba operaciones básicas, requería mayor valor agregado para su uso académico y técnico.

También era importante que el sistema tuviera persistencia real, pruebas automatizadas y documentación de endpoints.

6.2 Intervención Realizada

Se aplicaron varias mejoras perfectivas:

Persistencia real con SQLite

Se configuró el CRUD para guardar las misiones en una base de datos SQLite. Esto permite que los datos creados desde la interfaz se conserven después de recargar la página o reiniciar el servidor.

Mejoras en endpoints

Se implementaron endpoints REST para:

- Crear misiones.
- Listar misiones.
- Consultar una misión por ID.

- Actualizar misiones.
- Eliminar misiones.
- Filtrar resultados por estado, agencia o texto de búsqueda.

Documentación técnica

Se agregó documentación de endpoints mediante:

- Archivo OpenAPI.
- Colección Postman.
- Archivo de cambios aplicados.
- README del proyecto.

Pruebas automatizadas

Se incorporó una suite de pruebas para validar:

- Campos obligatorios.
- Entradas maliciosas.
- Seguridad con API-Key.
- Ciclo CRUD completo.
- Validación de estados.
- Rechazo de datos inválidos.

6.3 Justificación

Este cambio corresponde a **mantenimiento perfectivo** porque mejora funcionalidades existentes y aumenta el valor del sistema. No se limita a corregir errores, sino que optimiza la experiencia de uso, facilita las pruebas, mejora la documentación y hace que el CRUD sea más completo y profesional.

6.4 Resultado Obtenido

El sistema se volvió más funcional, documentado y fácil de mantener. La persistencia en SQLite permite trabajar con datos reales, mientras que las pruebas y documentación facilitan futuras modificaciones.

7. Mantenimiento Preventivo

7.1 Incidencia Técnica Encontrada

El sistema podía verse afectado por datos inválidos, entradas maliciosas o errores inesperados. Por ejemplo, se podían enviar campos vacíos, estados no permitidos, fechas con formato incorrecto o contenido peligroso como scripts e instrucciones SQL maliciosas.

7.2 Intervención Realizada

Se implementó programación defensiva mediante validaciones robustas.

Validaciones aplicadas:

Campo o entrada	Validación
name	Obligatorio y máximo 80 caracteres
agency	Obligatorio
type	Obligatorio
status	Solo permite planned, active, completed, failed o aborted
date	Formato YYYY-MM-DD y fecha válida
description	Límite de longitud
contenido malicioso	Bloqueo de patrones peligrosos
SQL Injection	Uso de consultas preparadas

También se bloquearon patrones como:

- <script>
- DROP TABLE
- UNION SELECT
- Intentos básicos de inyección SQL.

Además, se agregó:

- Middleware global de errores.
- Manejo de rutas no encontradas con respuesta 404.
- Función asyncRoute() para capturar errores en rutas asíncronas.
- Uso de consultas SQLite preparadas para evitar concatenación directa de SQL.

7.3 Justificación

Este cambio corresponde a **mantenimiento preventivo** porque reduce la posibilidad de fallos futuros, vulnerabilidades y comportamientos inesperados. El sistema valida los datos antes de procesarlos y maneja errores de manera controlada.

7.4 Resultado Obtenido

El CRUD es más robusto y seguro. Las validaciones reducen el riesgo de datos corruptos, errores en base de datos, ataques básicos de inyección y fallos no controlados.

8. Integración con EPN Event Manager

El CRUD fue integrado con el EPN Event Manager mediante solicitudes HTTP. Cada vez que se realiza una operación importante en el sistema, se envía un evento al hub.

Eventos enviados:

Acción en el CRUD	Evento enviado
Crear misión	CREATE
Consultar misiones	QUERY

Actualizar misión	UPDATE
Eliminar misión	DELETE

Esta integración permite mantener trazabilidad externa de las acciones realizadas en el CRUD.

La arquitectura general queda de la siguiente forma:

Usuario → Interfaz Missions CRUD → API Missions CRUD → SQLite
API Missions CRUD → EPN Event Manager → Base de datos de eventos

9. Pruebas Realizadas

Para validar el correcto funcionamiento del sistema se ejecutaron pruebas manuales y automáticas.

9.1 Pruebas manuales

Se verificó:

- Crear una misión desde la interfaz.
- Recargar la página y comprobar que la misión permanece guardada.
- Editar una misión.
- Eliminar una misión.
- Consultar registros desde SQLite.
- Verificar eventos enviados al EPN Event Manager.

9.2 Análisis estático

Se ejecutó ESLint para revisar el código sin necesidad de correr la aplicación.

Comando utilizado:

`npm run lint`

Resultado:

- El análisis se ejecutó correctamente.
- Se corrigieron inconsistencias detectadas.
- El código quedó sin errores reportados por ESLint.

9.3 Pruebas unitarias

Se ejecutaron pruebas automatizadas mediante Node:test.

Comando utilizado:

`npm test`

Resultado:

- 3 pruebas ejecutadas.
- 3 pruebas aprobadas.
- 0 pruebas fallidas.

Las pruebas validaron:

- Rechazo de misión sin campos obligatorios.
- Aceptación de una misión válida.
- Normalización de estados desconocidos.

9.4 Verificación de ejecución

También se verificó el funcionamiento del servidor con:

`npm start`

El sistema quedó disponible en:

`http://localhost:4000`

10. Evidencias del Funcionamiento

Las evidencias que se pueden presentar durante la sustentación son:

- Interfaz web del CRUD funcionando.
- Registro de misiones en la base SQLite.
- Consulta de endpoints REST.
- Envío de eventos al EPN Event Manager.
- Archivo de logs de auditoría.
- Ejecución de pruebas unitarias.
- Resultado exitoso de ESLint.
- Documentación OpenAPI.
- Colección Postman.

11. Conclusiones

El sistema Missions CRUD fue mejorado mediante la aplicación de los cuatro tipos de mantenimiento de software.

El mantenimiento correctivo permitió mejorar la trazabilidad y el manejo de errores. El mantenimiento adaptativo permitió configurar el sistema mediante variables de entorno y proteger los endpoints con API-Key. El mantenimiento perfectivo agregó persistencia real, documentación, pruebas y mejoras funcionales. Finalmente, el mantenimiento preventivo fortaleció el sistema frente a datos inválidos, entradas maliciosas y errores futuros.

Como resultado, el CRUD evolucionó de una aplicación básica a un sistema más robusto, seguro, mantenible y profesional, cumpliendo con los objetivos del taller de Construcción de Software.

12. Anexos

Endpoints principales

Método	Endpoint	Descripción
POST	/missions	Crear misión
GET	/missions	Listar misiones
GET	/missions/:id	Consultar misión por ID
PUT	/missions/:id	Actualizar misión
DELETE	/missions/:id	Eliminar misión

Comandos principales

Instalar dependencias:

```
npm install
```

Ejecutar servidor:

```
npm start
```

Ejecutar lint:

```
npm run lint
```

Ejecutar pruebas:

```
npm test
```

Header de seguridad

X-FIS-EPN-KEY: fis-epn-2025